
BCA EXAMINATION NOTES

Course Code: CA-125

CA-125 Web Design-II

Exam Pattern:

Q1. Attempt Any Six – 6 marks

Q2. Attempt Any Three – 6 marks

Q3. Attempt Any Two – 6 marks

Q4A. Attempt Any One – 4 marks | Q4B. Compulsory – 2 marks

Q5. Attempt Any One – 6 marks

UNIT 1: Introduction to Website Development & JavaScript

1.1 Website Development

Website development is the process of building and maintaining websites and web applications. It includes web design, content creation, client-side and server-side scripting, and network security configuration.

1.2 Website Development Phases

- Planning: Define purpose, target audience, scope, and requirements
- Design: Create layout, color scheme, navigation structure (wireframes/mockups)
- Development: Write HTML, CSS, JavaScript, server-side code
- Testing: Test functionality, compatibility, performance, security
- Deployment: Launch site on web server, configure DNS
- Maintenance: Regular updates, bug fixes, content updates

1.3 Website Authoring Tools

- Text Editors: Notepad, Sublime Text, VS Code — write code manually
- WYSIWYG Editors: Dreamweaver, Wix — visual drag-and-drop
- CMS: WordPress, Joomla — content management without coding
- IDEs: Eclipse, WebStorm — full development environments

1.4 Scripting Language

A scripting language is an interpreted programming language used to automate tasks or add interactivity. Unlike compiled languages, scripts are executed by an interpreter.

Types of Scripting Languages

- JavaScript: Client-side scripting for interactive web pages
- VBScript: Microsoft's client-side script (Internet Explorer only)
- ASP (Active Server Pages): Server-side scripting by Microsoft
- PHP (Hypertext Preprocessor): Open-source server-side scripting

1.5 Differences: Client-Side vs Server-Side Scripting

Execution: Client – On user's browser | Server – On web server

Language: Client – JavaScript, VBScript | Server – PHP, ASP, Node.js

Speed: Client – Faster (no server needed) | Server – Slower (needs server)

Security: Client – Less secure (code visible) | Server – More secure (hidden)

Purpose: Client – UI interaction, validation | Server – Database access, authentication

1.6 Introduction to JavaScript

JavaScript (JS) is a lightweight, interpreted, client-side scripting language used to make web pages interactive. It was created by Brendan Eich in 1995 at Netscape. JavaScript is case-sensitive and object-based.

1.7 Advantages of JavaScript

- Client-side execution: Fast, no server needed for simple tasks
- Enhanced interactivity: Dynamic content, animations, form validation
- Wide support: All modern browsers support JS natively

- Easy to learn: Simple syntax similar to C/Java
- Rich frameworks: React, Angular, Vue, jQuery available
- Versatile: Can also run on server (Node.js)
- Reduces server load: Client handles processing

1.8 Limitations of JavaScript

- Security: Code is visible to users; client-side scripts can be manipulated
- Browser dependency: May behave differently across browsers
- No file access: Cannot read/write local files (security restriction)
- No multi-threading: Single-threaded, blocking operations are slow
- Relies on browser: If JS disabled, functionality breaks

UNIT 2: Working with JavaScript

2.1 Embedding JavaScript in HTML

JavaScript can be embedded in three ways:

- Inline: `onclick='alert("Hi")'` — directly in HTML tag attribute
- Internal: Inside `<script>` tags in `<head>` or `<body>` section
- External: `src` attribute in `<script>` tag — `<script src='myfile.js'></script>`

2.2 Data Types in JavaScript

- Number: Integer or float. `var x = 10; var pi = 3.14;`
- String: Text in quotes. `var name = 'Ram';`
- Boolean: true or false. `var flag = true;`
- Undefined: Variable declared but not assigned. `var x;`
- Null: Intentional empty value. `var obj = null;`
- Object: Collection of key-value pairs. `var person = {name:'Ram', age:20};`
- Array: Ordered list. `var arr = [1,2,3];`

2.3 Variables

Variables store data. JavaScript uses `var` (function-scoped), `let` (block-scoped), and `const` (constant).

`var x = 10; let name = 'Ram'; const PI = 3.14;`

Rules: Must start with letter, `_`, or `$`. Case-sensitive. No spaces. No reserved words.

2.4 JavaScript Comments

Single-line: `// This is a comment`

Multi-line: `/* This is a multi-line comment */`

2.5 Operators and Expressions

- Arithmetic: `+` `-` `*` `/` `%` `++` `--`
- Assignment: `=` `+=` `-=` `*=` `/=` `%=`
- Comparison: `==` `===` `!=` `!==` `<` `>` `<=` `>=`
- Logical: `&&` `||` `!`
- Conditional (Ternary): `condition ? val1 : val2`

- **typeof:** Returns data type as string

2.6 Conditional Statements

if-else: `if(condition){ statements; } else { statements; }`

if-else if-else: `if(c1){} else if(c2){} else {}`

Switch: `switch(expr){ case v1: ...; break; case v2: ...; break; default: ...; }`

2.7 Looping Statements

for loop: `for(var i=0; i<10; i++){ statements; }`

while loop: `while(condition){ statements; }`

do-while: `do{ statements; } while(condition);`

UNIT 3: JavaScript Functions and Dialog Boxes

3.1 Introduction to Functions

A function is a reusable block of code that performs a specific task. Functions promote code reuse and modularity.

Syntax: `function functionName(parameters){ statements; return value; }`

3.2 Working with Functions

Defining: `function greet(name){ return 'Hello '+name; }`

Calling: `greet('Ram');` or `var result = greet('Sita');`

Anonymous function: `var add = function(a,b){ return a+b; };`

Arrow function (ES6): `var mul = (a,b) => a*b;`

Function with default parameters: `function greet(name='World'){ return 'Hello '+name; }`

3.3 Built-in String Functions

length: `str.length` — Returns length of string

toUpperCase(): `str.toUpperCase()` — Converts to uppercase

toLowerCase(): `str.toLowerCase()` — Converts to lowercase

trim(): `str.trim()` — Removes whitespace from both ends

indexOf(): `str.indexOf('word')` — Returns index of first occurrence

substring(): `str.substring(0,5)` — Extracts part of string

replace(): `str.replace('old','new')` — Replaces first occurrence

split(): `str.split(',')` — Splits string into array

charAt(): `str.charAt(0)` — Returns character at index

3.4 Dialog Boxes

Alert Dialog Box – `alert('message')`: Displays a message with OK button. No return value. Used for warnings/info.

Confirm Dialog Box – `confirm('question')`: Displays message with OK and Cancel. Returns true (OK) or false (Cancel).

Prompt Dialog Box – `prompt('message','default')`: Displays input field. Returns entered text or null if cancelled.

UNIT 4: JavaScript Events and Objects

4.1 JavaScript Events

- onclick:** Fires when element is clicked
- onmouseover:** Fires when mouse moves over element
- onmouseout:** Fires when mouse moves away from element
- onkeypress:** Fires when key is pressed while element has focus
- onkeydown:** Fires when key is pressed down
- onkeyup:** Fires when key is released
- onfocus:** Fires when element receives focus
- onblur:** Fires when element loses focus
- onload:** Fires when page or image has loaded
- onunload:** Fires when user leaves the page
- onsubmit:** Fires when form is submitted

4.2 String Methods (Object)

In JavaScript, strings are objects with built-in methods. Key methods: `concat()`, `slice()`, `includes()`, `startsWith()`, `endsWith()`, `repeat()`, `padStart()`, `padEnd()`

4.3 Math Object

- `Math.PI` – Value of π (3.14159...)
- `Math.round(x)` – Rounds to nearest integer
- `Math.ceil(x)` – Rounds up
- `Math.floor(x)` – Rounds down
- `Math.sqrt(x)` – Square root
- `Math.pow(x,y)` – x raised to y
- `Math.abs(x)` – Absolute value
- `Math.max(a,b)` – Maximum value
- `Math.min(a,b)` – Minimum value
- `Math.random()` – Random number between 0 and 1

4.4 Form Object

The Form object represents an HTML form. Access via `document.forms[0]` or `document.getElementById('formId')`.

Form properties: `action`, `method`, `name`, `elements[]`

Form methods: `submit()`, `reset()`

Form validation example: `document.getElementById('myForm').onsubmit = function(){ if(document.getElementById('name').value==""){ alert('Name required'); return false; } }`

DEFINITIONS (2-Mark Ready)

JavaScript: JavaScript is a lightweight, interpreted, client-side scripting language that adds interactivity and dynamic content to web pages.

Scripting Language: A scripting language is an interpreted programming language used to automate tasks or add interactivity to applications, executed line by line at runtime.

Event: An event is an action or occurrence (like mouse click, key press, page load) that the browser can detect and JavaScript can respond to.

Function: A function is a reusable, named block of JavaScript code that performs a specific task and can be called multiple times.

Alert Box: An alert box is a JavaScript dialog (alert()) that displays a message to the user with an OK button, pausing script execution until dismissed.

Variable: A variable is a named storage location in memory that holds a value. In JavaScript, declared using var, let, or const.

Client-Side Scripting: Client-side scripting refers to scripts executed by the user's browser without server interaction, used for UI interactivity and validation.

Math Object: The Math object is a built-in JavaScript object providing mathematical constants (Math.PI) and methods (Math.round, Math.sqrt, etc.)

Form Validation: Form validation is the process of checking user input in a form using JavaScript before submitting to the server, ensuring correct/complete data.

String: A string in JavaScript is a sequence of characters enclosed in single or double quotes, representing text data.

20 MOST IMPORTANT QUESTIONS WITH ANSWERS

Q1. What is JavaScript? State its advantages and limitations. [6M]

JavaScript is a lightweight, interpreted, object-based scripting language used to create interactive and dynamic web pages. Created by Brendan Eich in 1995.

Advantages:

1. Executes in browser – no server required for basic tasks
2. Makes pages interactive (animations, validation, dynamic content)
3. Supported by all modern browsers
4. Easy to learn – simple C/Java-like syntax
5. Rich ecosystem – React, Angular, jQuery frameworks
6. Can run on server too (Node.js)

Limitations:

1. Security – code visible in browser; can be exploited
2. Browser dependency – behavior may vary across browsers
3. No file system access – can't read/write local files directly
4. Single-threaded – cannot perform true parallel processing
5. Functionality breaks if JS is disabled

Q2. Differentiate client-side and server-side scripting. [4M]

CLIENT-SIDE SCRIPTING:

1. Executes in user's web browser
2. Languages: JavaScript, VBScript
3. Fast – no round-trip to server

4. Used for: UI interactivity, form validation, animations
 5. Security: Code is visible to users
- Example: form.onsubmit validation

SERVER-SIDE SCRIPTING:

1. Executes on web server
 2. Languages: PHP, ASP, Node.js, Python
 3. Slower – requires server round-trip
 4. Used for: Database access, user authentication, file operations
 5. Security: Code hidden from users
- Example: Login validation against database

Q3. Explain data types in JavaScript with examples. [4M]

JavaScript has 7 primitive data types:

1. Number: Integer and float. var x=10; var pi=3.14;
2. String: Text. var name='Ram'; var msg="Hello";
3. Boolean: true or false. var flag=true;
4. Undefined: Declared but not assigned. var x;
5. Null: Intentionally empty. var obj=null;
6. BigInt: Very large integers. var big=9007n;
7. Symbol: Unique identifiers. var sym=Symbol('id');

Composite types:

8. Object: Key-value pairs. var p={name:'Ram',age:20};
9. Array: Ordered list. var arr=[1,2,3];

Q4. Explain conditional statements in JavaScript. [6M]

1. if STATEMENT:

```
if(condition) { statements; }
```

Example:

```
if(marks>=40) { alert('Pass'); }
```

2. if-else:

```
if(condition){ statements; }
```

```
else { statements; }
```

Example:

```
if(num>0) alert('Positive');
```

```
else alert('Non-positive');
```

3. if-else if-else LADDER:

```
if(c1){ ... }
```

```
else if(c2){ ... }
```

```
else if(c3){ ... }
```

```
else { ... }
```

Example:

```
if(m>=90) grade='A';
```

```
else if(m>=75) grade='B';
```

```
else if(m>=60) grade='C';
```

```
else grade='F';
```

4. SWITCH STATEMENT:

```
switch(expression){  
  case v1: statements; break;  
  case v2: statements; break;  
  default: statements;  
}
```

Example:

```
switch(day){  
  case 1: alert('Monday'); break;  
  case 2: alert('Tuesday'); break;  
  default: alert('Other');  
}
```

Q5. Explain loops in JavaScript with examples. [6M]

1. FOR LOOP: Used when number of iterations is known.

```
for(var i=1; i<=5; i++){  
  document.write(i+' ');  
}
```

Output: 1 2 3 4 5

2. WHILE LOOP: Used when iterations are unknown.

```
var i=1;  
while(i<=5){  
  document.write(i+' ');  
  i++;  
}
```

3. DO-WHILE LOOP: Executes body at least once.

```
var i=1;  
do{  
  document.write(i+' ');  
  i++;  
} while(i<=5);
```

4. FOR...IN LOOP: Iterates over object properties.

```
var person={name:'Ram',age:20};  
for(var key in person){  
  document.write(key+' '+person[key]);  
}
```

5. FOR...OF LOOP: Iterates over array elements.

```
var arr=[10,20,30];  
for(var val of arr){  
  document.write(val+' ');  
}
```

Q6. What are functions in JavaScript? Explain with examples. [6M]

A function is a reusable block of code that performs a specific task.

Syntax: function name(params){ body; return val; }

1. Regular Function:

```
function add(a,b){  
  return a+b;  
}  
var result = add(3,4); // result=7
```

2. Function with No Return:

```
function greet(name){  
  alert('Hello '+name);  
}  
greet('Ram');
```

3. Anonymous Function:

```
var square = function(x){ return x*x; };  
alert(square(5)); // 25
```

4. Arrow Function (ES6):

```
var mul = (a,b) => a*b;  
alert(mul(3,4)); // 12
```

5. Self-invoking Function:

```
(function(){  
  alert('I run immediately!');  
})();
```

Advantages: Code reuse, modularity, easier debugging.

Q7. Explain built-in string functions in JavaScript. [6M]

JavaScript string is an object with many built-in methods:

1. length: var s='Hello'; s.length; // 5
2. toUpperCase(): 'hello'.toUpperCase(); // 'HELLO'
3. toLowerCase(): 'HELLO'.toLowerCase(); // 'hello'
4. indexOf(): 'Hello World'.indexOf('World'); // 6
5. substring(start,end): 'Hello World'.substring(0,5); // 'Hello'
6. slice(start,end): 'Hello World'.slice(6); // 'World'
7. replace(): 'Hello Ram'.replace('Ram','Sita'); // 'Hello Sita'
8. split(): 'a,b,c'.split(','); // ['a','b','c']
9. trim(): ' Hello '.trim(); // 'Hello'
10. charAt(): 'Hello'.charAt(1); // 'e'
11. includes(): 'Hello'.includes('ell'); // true
12. concat(): 'Hello'.concat(' World'); // 'Hello World'

Q8. Explain dialog boxes in JavaScript with examples. [4M]

JavaScript provides three dialog boxes:

1. ALERT BOX:

```
alert('This is an alert!');
```

Displays message with OK button only.
No return value. Blocks further execution.
Use: Show warnings, info to user.

2. CONFIRM BOX:

```
var result = confirm('Are you sure?');  
if(result==true){ alert('You confirmed!'); }  
else { alert('You cancelled!'); }  
Returns true (OK clicked) or false (Cancel clicked).  
Use: Get user confirmation before action.
```

3. PROMPT BOX:

```
var name = prompt('Enter your name:', 'Default');  
if(name!=null){ alert('Hello '+name); }  
Displays input field.  
Returns entered text or null if cancelled.  
Use: Get simple text input from user.
```

Q9. Explain JavaScript events with examples. [6M]

Events are actions/occurrences that JavaScript can detect and respond to.

1. onclick: When element is clicked.

```
<button onclick='alert("Clicked!")'>Click</button>
```

2. onmouseover: When mouse hovers over element.

```
<p onmouseover='this.style.color="red"'>Hover me</p>
```

3. onmouseout: When mouse leaves element.

```
<p onmouseout='this.style.color="black"'>Out</p>
```

4. onkeydown: When key is pressed.

```
<input onkeydown='console.log(event.key)'>
```

5. onload: When page loads.

```
<body onload='alert("Page loaded!")'>
```

6. onsubmit: When form is submitted.

```
<form onsubmit='return validate()'>
```

7. onfocus / onblur:

```
<input onfocus='this.style.background="yellow"'  
onblur='this.style.background="white"'>
```

Q10. Explain Math object in JavaScript. [4M]

The Math object provides mathematical functions and constants in JavaScript.

Constants:

Math.PI = 3.14159...

Math.E = 2.71828...

Methods:

1. `Math.round(4.6) = 5` (nearest integer)
2. `Math.ceil(4.1) = 5` (always rounds up)
3. `Math.floor(4.9) = 4` (always rounds down)
4. `Math.sqrt(16) = 4` (square root)
5. `Math.pow(2,3) = 8` (2 to the power 3)
6. `Math.abs(-5) = 5` (absolute value)
7. `Math.max(3,7,1) = 7` (maximum)
8. `Math.min(3,7,1) = 1` (minimum)
9. `Math.random() = 0.something` (0 to <1)
10. Random int 1-10: `Math.floor(Math.random()*10)+1`

Q11. Explain website development phases. [4M]

A website goes through these development phases:

1. **PLANNING:** Define goals, audience, content, budget, timeline.
2. **DESIGN:** Create wireframes, mockups, color scheme, navigation.
3. **DEVELOPMENT:** Code HTML/CSS structure, JavaScript interaction, server-side logic.
4. **CONTENT CREATION:** Write text, prepare images, create media.
5. **TESTING:** Cross-browser testing, mobile responsiveness, security checks.
6. **DEPLOYMENT:** Upload to web server, configure domain and DNS.
7. **MAINTENANCE:** Regular updates, bug fixes, performance monitoring.

Q12. Explain how to embed JavaScript in HTML. [4M]

JavaScript can be embedded in HTML in three ways:

1. **INLINE SCRIPTING** (Inside HTML attributes):

```
<button onclick='alert("Hello!")'>Click</button>
```

Used for simple, short scripts.

2. **INTERNAL SCRIPTING** (Inside `<script>` tags):

```
<head>
```

```
<script type='text/javascript'>
```

```
function greet(){ alert("Welcome!"); }
```

```
</script>
```

```
</head>
```

Used when script is specific to one page.

3. **EXTERNAL SCRIPTING** (Separate .js file):

```
<script src='myScript.js'></script>
```

Used when script is used across multiple pages.

Advantages: Cleaner code, caching, reusability.

Q13. What is the Form object in JavaScript? [4M]

The Form object represents an HTML `<form>` element in JavaScript.

Accessing: `document.forms[0]` or `document.getElementById('myForm')`

Properties:

- `action`: URL where form data is sent

- method: 'get' or 'post'
- name: Name of the form
- elements: Array of form controls

Methods:

- submit(): Submits the form programmatically
- reset(): Resets all form fields

Form Validation Example:

```
function validate(){
var name = document.getElementById('name').value;
if(name.trim()==""){
alert('Name is required!');
return false; // prevent submission
}
return true; // allow submission
}
```

Q14. What are operators in JavaScript? [4M]

JavaScript operators perform operations on values:

1. ARITHMETIC: + - * / % ++ --
5+3=8, 10%3=1, ++i (pre-increment)
2. ASSIGNMENT: = += -= *= /= %=
x+=5 means x=x+5
3. COMPARISON: == === != !== < > <= >=
== checks value; === checks value AND type
4. LOGICAL: && (AND) || (OR) ! (NOT)
true&&false=false, !true=false
5. TERNARY: condition ? val1 : val2
var result = (x>0) ? 'positive' : 'non-positive';
6. typeof: typeof 42 = 'number'
7. String concatenation: 'Hello' + ' ' + 'World' = 'Hello World'

Q15. Explain scripting languages in web development. [4M]

Scripting languages add dynamic behavior to web pages:

1. JAVASCRIPT:

Client-side scripting. Runs in browser.

Used for: Form validation, animations, AJAX.

Example: onclick, document.getElementById

2. VBSCRIPT:

Microsoft's client-side script.

Only works in Internet Explorer.

Similar syntax to VBA.

3. ASP (Active Server Pages):

Server-side by Microsoft.

Runs on IIS web server.
Generates HTML dynamically.

4. PHP (PHP: Hypertext Preprocessor):
Open-source server-side script.
Runs on Apache/Nginx.
Commonly used with MySQL for database-driven websites.
Example: `<?php echo 'Hello World'; ?>`

Difference: Client-side runs in browser; server-side runs on server and sends HTML.

Q16. What is switch-case in JavaScript? Write an example. [4M]

Switch-case tests a variable against multiple values.

Syntax:

```
switch(expression){  
  case value1: statements; break;  
  case value2: statements; break;  
  default: default_statements;  
}
```

Example – Day name:

```
var day = 3;  
switch(day){  
  case 1: alert('Monday'); break;  
  case 2: alert('Tuesday'); break;  
  case 3: alert('Wednesday'); break;  
  case 4: alert('Thursday'); break;  
  case 5: alert('Friday'); break;  
  default: alert('Weekend');  
}
```

Output: Wednesday

Rules: break prevents fall-through to next case. default executes when no case matches. Case values must be constants.

Q17. Write a JavaScript program to validate a form. [4M]

HTML Form with JavaScript Validation:

```
<form name='myForm' onsubmit='return validate()'>  
  Name: <input type='text' id='name'><br>  
  Email: <input type='text' id='email'><br>  
  <input type='submit' value='Submit'>  
</form>
```

```
<script>  
function validate(){  
  var name = document.getElementById('name').value;  
  var email = document.getElementById('email').value;
```

```
  if(name.trim()==""){
```

```

alert('Name is required!');
return false;
}
if(email.indexOf('@')== -1){
alert('Invalid email!');
return false;
}
return true; // form submits
}
</script>

```

Q18. What are the key events in JavaScript? [2M]

Key events detect keyboard interactions:

1. onkeydown: Fires when key is pressed down (fires continuously while held)
2. onkeypress: Fires after keydown when a character key is pressed (deprecated in modern JS)
3. onkeyup: Fires when key is released

Example:

```

<input onkeyup='showLength(this)'>
function showLength(el){
document.getElementById('count').innerHTML = el.value.length + ' chars';
}

```

event.key gives key name. event.keyCode gives key code (deprecated; use event.key instead).

Q19. What are the advantages of JavaScript over static HTML? [2M]

JavaScript advantages over static HTML:

1. Interactivity: HTML is static; JS adds click, hover, keyboard responses
2. Form Validation: JS validates input before sending to server
3. Dynamic Content: Update page content without reloading (AJAX)
4. Animations: Create visual effects, slideshows, transitions
5. Conditional Logic: Show/hide elements based on conditions
6. Mathematical Operations: Perform calculations in browser
7. DOM Manipulation: Change any HTML element dynamically

Q20. Explain the confirm() and prompt() functions. [4M]

CONFIRM DIALOG BOX:

Syntax: var result = confirm('Question?');

Displays: Message with OK and Cancel buttons.

Return value: true (OK clicked), false (Cancel clicked).

Example:

```

var del = confirm('Delete this record?');
if(del){
alert('Record deleted.');
```

```

} else {
alert('Cancelled.');
```

```

}

```

Use: Get user's yes/no decision before performing an action.

PROMPT DIALOG BOX:

Syntax: `var input = prompt('Message', 'Default value');`

Displays: Message + text input field.

Return value: String entered by user, or null if cancelled.

Example:

```
var age = prompt('Enter your age:', '18');
```

```
if(age != null){
```

```
  alert('You are ' + age + ' years old.');
```

```
}
```

Use: Collect simple text input from user.